GenAI / AI Agent / RAG Feature Plan

InfraFlow — a phased roadmap grounded in what already exists in the codebase, not a generic wishlist.

CURRENT STATE — WHAT ALREADY EXISTS

A RAG agent already exists in the codebase: an "AWS Well-Architected Framework" chat with source citations, full conversation history, and workspace-scoped role-based access — but it is currently disabled behind a "coming soon" gate in the dashboard. It proxies to an external FastAPI microservice (**RAG_API_URL** / `/ask`) that is not part of this repository and is not running. No LLM API (OpenAI, Anthropic, etc.) is wired in anywhere else in the codebase. The conversation data model already captures `diagramId` and `awsAccountId` context, but that context is never actually used in the retrieval call today — answers are generic, not personalized to the user's own infrastructure. This gap drives most of the prioritization below.

Phase 1 — Finish and personalize what already exists

The highest-leverage move is not new infrastructure — it is making the existing RAG agent actually useful.

- **Ground it in the user's own diagram**, not just a static PDF. When a conversation has a diagramId, inject the current nodes/edges, active validation issues, and cost estimates into the prompt alongside retrieved Well-Architected passages, so "why is this flagged?" answers about the user's own VPC, not AWS in the abstract.
- **Replace the external FastAPI dependency** with an in-repo retrieval service (or automate standing it up) — right now the feature is invisible to anyone who does not know to run a separate Python process.
- **Wire an actual LLM** (Claude via the Messages API is the natural fit given the existing structured-context pattern) for generation, with the vector store handling retrieval only.

Phase 2 — Turn the agent from Q&A into action (agentic / tool-use)

This is where "AI agent" stops meaning "chatbot" and starts meaning something that changes the diagram.

- **Tool-calling over the diagram store:** expose addServiceNode, updateNodeConfig, addGroupNode, and exportTerraform as callable tools. "Add a Multi-AZ RDS Postgres behind the private subnet" → the agent plans and executes; the user approves the diff before it applies, matching the existing validate-before-deploy pattern.
- **"Generate Terraform fix":** already a suggested-action button in the UI, not yet wired to anything real. Feed the current exportTerraform() output plus validateDiagram() issues to the LLM, get back a patch, show a diff, and let the user accept or reject each change.
- **Natural-language → diagram generation:** "3-tier web app with an ALB, ECS Fargate, and RDS" → the LLM emits a structured node/edge graph using the same shape as the existing templateSnapshot() builders, dropped onto the canvas as a starting point.

Phase 3 — Specialized agents over data already collected

Cost, security, and drift data are already pulled into the app today — none of it is summarized by AI yet.

- **Cost agent:** narrate getBillingRealtimeInsights trends ("EC2 spend is up 40% this week because...") instead of raw charts.
- **Security agent:** turn validateDiagram's security findings into a prioritized, plain-English remediation plan.
- **Drift agent:** when deployed state diverges from the diagram, explain what changed and why it matters, not just flag it.

Phase 4 — Latest-tech plumbing

- **Streaming responses** (Server-Sent Events) for the chat UI — currently request/response only.
- **Semantic search over the service palette** — useful once the AWS service catalog grows past what substring-match search can handle well.
- **MCP-style tool boundary** if the agent should be extensible by others later, rather than a hardcoded tool list.

Performance (separate from AI, as requested)

The production build has flagged this on every build this session: a main bundle of roughly 1.7 MB, plus a 1.4 MB ELK layout chunk. Concrete fixes:

- Lazy-load Monaco (Terraform preview), ELK, and dagre — none are needed on first paint.
- Code-split rarely-visited dashboard pages (Super Admin, Support) the same way.
- For very large diagrams (300+ nodes, a target set early in this project), add node virtualization beyond what onlyRenderVisibleElements already provides.
- The backend runs on Lambda — if agent responses need to feel snappy, provisioned concurrency avoids cold-start latency specifically on that request path.

At a glance

Phase	Theme	Effort	Depends on
1	Personalize existing RAG agent	Medium	Stand up retrieval service + LLM API key
2	Agentic tool-use over the diagram	High	Phase 1 (LLM wiring)
3	Specialized cost/security/drift agents	Medium	Phase 1
4	Streaming, semantic search, MCP boundary	Medium	Phase 1-2
-	Bundle-size / performance pass	Low-Medium	Independent — can start anytime

---

Generated from a direct review of the current codebase (agentApi.ts, AgentPage.tsx, agentController.js, agentResponder.js) — reflects what is actually implemented today, not aspirational planning.